CHAPTER 9

STRING HANDLING AND IO FUNCTIONS

CHAPTER ESSENCE AT A GLANCE:
Master null-terminated character array manipulation and stream buffer operations spanning character classification, string concatenation, copying, comparison, and multi-word input handling.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Null Terminator Contract:** C++ represents strings as character arrays terminated by an invisible null byte ('\0', ASCII 0) marking the boundary of valid string data.

• **Whitespace Delimited Extraction Limitation:** Standard stream extraction cin >> halts parsing at whitespace, necessitating cin.getline() to ingest sentences containing spaces safely.

• **Direct Memory String Functions:** Standard library functions in <cstring> iterate across strings until encountering the null terminator to compute lengths, copy buffers, or compare lexical order.

• **Character Classification Predicates:** Header <cctype> provides robust non-zero boolean predicates to test whether individual character byte codes are alphabetic, numeric, or uppercase.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{char str}[N] \rightarrow \text{Max Characters} = N - 1 \quad (\text{Last byte is } '\backslash 0')$$

Null-Terminated String Buffer Sizing

$$\text{cin.getline}(\text{buffer}, \text{size}) \quad (\text{Reads lines containing spaces})$$

Multi-Word Stream Input Function

$$\text{strcmp}(s_1, s_2) \rightarrow 0 \ (\text{Equal}), \ < 0 \ (s_1 < s_2), \ > 0 \ (s_1 > s_2)$$

Lexicographical String Comparison Output

$$\text{strlen}(\text{str}) \rightarrow \text{Character Count excluding } '\backslash 0'$$

String Length Determination Metric

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Always dimension character arrays at least one byte larger than the expected text length to accommodate the terminating null character '\0'.

• **Alert:** strcpy(dest, src) copies src into dest; ensure dest has sufficient allocated memory buffer capacity to prevent severe buffer overflow errors.

• **Alert:** strcmp() returns 0 when two strings are exactly identical; do not treat 0 as boolean false! Always check 'if (strcmp(s1, s2) == 0)'.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What is the importance of the null character ('\0') in C++ strings? [Score: 2 Marks]

Answer & Key Valuation Steps:

• **String Delimiter Role:** The null character ('\0', ASCII code 0) marks the end of a valid character string stored in an array. **[1 Mark]**

• **Library Function Safety:** String functions such as strlen(), strcpy(), and stream output cout rely on '\0' to know when to cease memory reading. **[1 Mark]**

2. Why is cin.getline() preferred over cin >> for reading string inputs? [Score: 2 Marks]

Answer & Key Valuation Steps:

• **cin >> Limitation:** The standard extraction operator stops reading at the first encountered whitespace character (space, tab, or newline), reading only single words. **[1 Mark]**

• **cin.getline() Capability:** Reads an entire line of text including embedded blank spaces until a designated delimiter or newline is reached. **[1 Mark]**

3. Explain any three string manipulation functions declared in header <cstring>: strlen(), strcpy(), and strcat(). [Score: 3 Marks]

Answer & Key Valuation Steps:

• **strlen(s):** Computes and returns the exact number of characters in string s excluding the terminating null character. **[1 Mark]**

• **strcpy(dest, src):** Copies the character sequence from src string (including null character) into destination array dest. **[1 Mark]**

• **strcat(dest, src):** Appends a copy of src string to the end of dest string, replacing dest's null character and adding a new null at end. **[1 Mark]**

4. Explain character classification functions declared in <cctype>: isalpha(), isdigit(), isalnum(), and toupper(). [Score: 3 Marks]

Answer & Key Valuation Steps:

• **isalpha(c) & isdigit(c):** isalpha checks if c is an alphabetic letter (A-Z, a-z); isdigit checks if c is a decimal digit (0-9). Both return non-zero for true, 0 for false. **[1 Mark]**

• **isalnum(c):** Checks if c is alphanumeric (either an alphabet letter or a digit). **[1 Mark]**

• **toupper(c):** Converts lowercase letter c to uppercase equivalent; leaves non-alphabets and uppercase letters unchanged. **[1 Mark]**

5. Explain character stream functions cin.get() and cout.put() with syntax examples. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **cin.get(ch):** Reads a single character from standard input stream, including whitespace characters like space or newline, storing it in ch. **[1.5 Marks]**

• **cout.put(ch):** Inserts and displays a single character ch onto standard output screen (e.g. cout.put('A');). **[1.5 Marks]**

6. Write a complete C++ program to check whether a user-entered string is a Palindrome without using library reversal functions. [Score: 4 Marks]

Answer & Key Valuation Steps:

• **Input & Length:** #include <iostream>n#include <cstring>vsing namespace std;nint main() { char str[100]; cin.getline(str, 100); int len = strlen(str); bool isPalin = true; **[1.5 Marks]**

• **Two-Pointer Comparison:** for (int i = 0; i * 2 < len; i++) { if (str[i] != str[len - 1 - i]) { isPalin = false; break; } } **[1.5 Marks]**

• **Output & Boxed Result:** if (isPalin) cout << 'Palindrome'; else cout << 'Not Palindrome'; return 0; } Boxed Result: Symmetrical character verification. **[1 Mark]**

7. Write a C++ program to count the total number of vowels, consonants, and digits present in a line of text. **[Score: 3 Marks]**

Answer & Key Valuation Steps:

• **Setup & Traversal:** char str[100]; cin.getline(str, 100); int v=0, c=0, d=0; for(int i=0; str[i] ≠ '\0'; i++) { char ch = tolower(str[i]); **[1 Mark]**

• **Classification Logic:** if(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u') v++; else if(isalpha(ch)) c++; else if(isdigit(ch)) d++; } **[1.5 Marks]**

• **Display & Boxed Result:** cout << 'Vowels: ' << v << ' Consonants: ' << c << ' Digits: ' << d; Boxed Result: Traversal parsing via cctype predicates. **[0.5 Mark]**